

CO4-AT-2

MLA0201-Fundamentals Of Machine Learning
Industry Problem Solving Task

QUESTION-1:**Fraud Detection Using Bayesian Networks:****Industry Scenario**

A digital banking organization wants to identify whether a credit-card transaction is **Fraudulent** or **Legitimate**.

The available attributes are:

- Transaction Amount
- Transaction Location
- Customer Login Device
- Previous Fraud History
- Fraudulent Transaction — target variable

A **Bayesian Network (BN)** is suitable because it represents probabilistic relationships between these variables and calculates the probability of fraud based on observed transaction information.

The relationships are:

$$P(F, r, a, ud \mid Amount Location Device History)$$

The Bayesian Network calculates the probability of:

- **Fraud = Yes**
- **Fraud = No**

based on the customer's transaction details.

Conditional Probability Tables (CPTs)

For simplicity, the variables are divided into two categories.

Transaction Amount

Amount	Probability
Low	0.60
High	0.40

Transaction Location

Location	Probability
Known	0.70
Unknown	0.30

Login Device

Device	Probability
Trusted	0.75
Untrusted	0.25

Previous Fraud History

History	Probability
No	0.80
Yes	0.20

Fraud CPT

Amount	Location	Device	History	P(Fraud=Yes)
Low	Known	Trusted	No	0.01
Low	Known	Trusted	Yes	0.10
Low	Known	Untrusted	No	0.05
Low	Known	Untrusted	Yes	0.30
Low	Unknown	Trusted	No	0.08
Low	Unknown	Trusted	Yes	0.35
Low	Unknown	Untrusted	No	0.20
Low	Unknown	Untrusted	Yes	0.60
High	Known	Trusted	No	0.05
High	Known	Trusted	Yes	0.25
High	Known	Untrusted	No	0.15
High	Known	Untrusted	Yes	0.50
High	Unknown	Trusted	No	0.20
High	Unknown	Trusted	Yes	0.55
High	Unknown	Untrusted	No	0.45
High	Unknown	Untrusted	Yes	0.85

The probability of **No Fraud** is:

$$P(\text{NoFraud}) = 1 - P(\text{Fraud})$$

Code:

```
# Import libraries
from pgmpy.models import DiscreteBayesianNetwork
from pgmpy.factors.discrete import TabularCPD
from pgmpy.inference import VariableElimination
print("Libraries imported successfully!")
model = DiscreteBayesianNetwork([
    ("Amount", "Fraud"),
    ("Location", "Fraud"),
    ("Device", "Fraud"),
    ("History", "Fraud")
])
print("\nBayesian Network created successfully!")
cpd_amount = TabularCPD(
    variable="Amount",
    variable_card=2,
    values=[
        [0.60], # Low
        [0.40] # High
    ],
    state_names={
        "Amount": ["Low", "High"]
    }
)
```

```

cpd_location = TabularCPD(
    variable="Location",
    variable_card=2,
    values=[
        [0.70], # Known
        [0.30] # Unknown
    ],
    state_names={
        "Location": ["Known", "Unknown"]
    }
)

cpd_device = TabularCPD(
    variable="Device",
    variable_card=2,
    values=[
        [0.75], # Trusted
        [0.25] # Untrusted
    ],
    state_names={
        "Device": ["Trusted", "Untrusted"]
    }
)

cpd_history = TabularCPD(
    variable="History",
    variable_card=2,
    values=[
        [0.80], # No
        [0.20] # Yes
    ],
    state_names={
        "History": ["No", "Yes"]
    }
)

cpd_fraud = TabularCPD(

```

```

    variable="Fraud",
    variable_card=2,

    values=[
        # Fraud = No
        [
            0.99, 0.90, 0.95, 0.70,
            0.92, 0.65, 0.80, 0.40,
            0.95, 0.75, 0.85, 0.50,
            0.80, 0.45, 0.55, 0.15
        ],

        # Fraud = Yes
        [
            0.01, 0.10, 0.05, 0.30,
            0.08, 0.35, 0.20, 0.60,
            0.05, 0.25, 0.15, 0.50,
            0.20, 0.55, 0.45, 0.85
        ]
    ],
    evidence=[
        "Amount",
        "Location",
        "Device",
        "History"
    ],
    evidence_card=[
        2, 2, 2, 2
    ],

    state_names={
        "Fraud": ["No", "Yes"],
        "Amount": ["Low", "High"],
        "Location": ["Known", "Unknown"],
        "Device": ["Trusted", "Untrusted"],
        "History": ["No", "Yes"]
    }
)

```

```

print("\nNetwork Relationships:")
print("Amount ---> Fraud")
print("Location ---> Fraud")
print("Device ---> Fraud")
print("History ---> Fraud")
inference = VariableElimination(model)
transaction = {
    "Amount": "High",
    "Location": "Unknown",
    "Device": "Untrusted",
    "History": "Yes"
}

result = inference.query(
    variables=["Fraud"],
    evidence=transaction
)

print("\n" + "="*50)
print("TRANSACTION DETAILS")
print("="*50)
print("Transaction Amount :", transaction["Amount"])
print("Transaction Location:", transaction["Location"])
print("Login Device      :", transaction["Device"])
print("Previous Fraud    :", transaction["History"])
prob_no_fraud = result.values[0]
prob_fraud = result.values[1]
print("\n" + "="*50)
print("FRAUD PROBABILITY")
print("="*50)
print(f"Probability of Legitimate Transaction : "
      f"{prob_no_fraud*100:.2f}%")

print(f"Probability of Fraudulent Transaction : "
      f"{prob_fraud*100:.2f}%")
print("\n" + "="*50)
print("FINAL PREDICTION")
print("="*50)

if prob_fraud >= 0.50:
    print("🚨 FRAUDULENT TRANSACTION")
    print("Action: Transaction should be flagged for review.")
else:
    print("✅ LEGITIMATE TRANSACTION")
    print("Action: Transaction can be approved.")
print("\n" + "="*50)
print("INTERPRETATION")
print("="*50)

if prob_fraud >= 0.50:
    print(
        "The transaction has a high probability of fraud because "
        "the transaction amount is high, the location is unknown, "
        "the device is untrusted, and previous fraud history exists."
    )
else:
    print(
        "The transaction has a lower probability of fraud based "
        "on the given transaction characteristics."
    )

```

Output:

```

... Libraries imported successfully!

Bayesian Network created successfully!

Checking Bayesian Network...
Bayesian Network is VALID!

Network Relationships:
Amount ---> Fraud
Location ---> Fraud
Device ---> Fraud
History ---> Fraud

=====
TRANSACTION DETAILS
=====
Transaction Amount : High
Transaction Location: Unknown
Login Device      : Untrusted
Previous Fraud    : Yes

=====
FRAUD PROBABILITY
=====
Probability of Legitimate Transaction : 15.00%
Probability of Fraudulent Transaction : 85.00%

=====
FINAL PREDICTION
=====
🚨 FRAUDULENT TRANSACTION
Action: Transaction should be flagged for review.

=====
INTERPRETATION
=====
The transaction has a high probability of fraud because the transaction amount is high, the location is unknown, the device is untrusted, and previous fraud history

```

Result:

Unlike some complex black-box models, Bayesian Networks provide a clear probabilistic relationship between the input variables and the fraud prediction.

Conclusion:

A Bayesian Network provides an effective probabilistic approach for credit-card fraud detection. By representing relationships between transaction characteristics and fraud history, the system can calculate the probability that a new transaction is fraudulent. In the given example, the combination of a high transaction amount, unknown location, untrusted device, and previous fraud history produces a high fraud probability, so the transaction is classified as fraudulent.

QUESTION-2:**Industry Scenario**

A manufacturing company continuously monitors machines using sensors such as:

- **Temperature**
- **Vibration**
- **Motor Noise**

The actual condition of the machine cannot be directly observed. Therefore, the machine condition is considered a hidden state.

The three hidden states are:

- **Healthy**
- **Warning**
- **Failure**

The sensor readings are the **observations**.

The HMM learns the relationship between observations and hidden machine conditions and predicts the most likely condition sequence.

Hidden States

$$S = \{H, e, althyWarningFailure\}$$

Transition Probabilities:

The transition matrix describes how likely the machine is to move from one condition to another.

From / To	Healthy	Warning	Failure
Healthy	0.80	0.18	0.02
Warning	0.15	0.65	0.20
Failure	0.05	0.10	0.85

For example:

$$P(Warning | Healthy) = 0.18$$

means there is an 18% probability that a healthy machine moves into a warning state.

Emission Probabilities:

Emission probabilities describe the probability of observing a sensor condition for each hidden machine state.

Machine State	Normal	High	Critical
Healthy	0.85	0.14	0.01
Warning	0.20	0.65	0.15
Failure	0.02	0.18	0.80

For example:

$$P(\text{Critical} \mid \text{Failure}) = 0.80$$

This means a machine in the failure state has an 80% probability of producing critical sensor observations.

Code:

```
# =====  
# PREDICTIVE MACHINE MAINTENANCE USING HIDDEN MARKOV MODEL  
# =====  
!pip -q install hmmlearn  
import numpy as np  
from hmmlearn import hmm  
print("Libraries imported successfully!")  
# =====  
# 1. DEFINE HIDDEN STATES  
# =====  
states = {0: "Healthy", 1: "Warning", 2: "Failure"}  
print("\nHidden States:")  
for key, value in states.items():  
    print(key, "=", value)  
# =====  
# 2. CREATE HIDDEN MARKOV MODEL  
# =====  
model = hmm.CategoricalHMM(n_components=3, random_state=42)  
# =====  
# 3. INITIAL STATE PROBABILITIES  
# =====  
model.startprob_ = np.array([0.70, 0.25, 0.05])  
# =====  
# 4. TRANSITION PROBABILITY MATRIX  
# =====  
model.transmat_ = np.array([  
    [0.80, 0.18, 0.02],  
    [0.15, 0.65, 0.20],  
    [0.05, 0.10, 0.85]  
)  
# =====  
# 5. EMISSION PROBABILITY MATRIX  
# =====  
# 0 = Normal, 1 = High, 2 = Critical  
model.emissionprob_ = np.array([  
    [0.85, 0.14, 0.01],  
    [0.20, 0.65, 0.15],  
    [0.02, 0.18, 0.80]  
)  
# =====  
# 6. SENSOR OBSERVATION SEQUENCE  
# =====  
observations = np.array([[0], [0], [1], [1], [2], [2]])  
print("\nSensor Observation Sequence:")  
print("Normal → Normal → High → High → Critical → Critical")  
# =====  
# 7. PREDICT HIDDEN STATES  
# =====  
hidden_states = model.predict(observations)  
# =====  
# 8. DISPLAY PREDICTED STATES  
# =====  
print("\n" + "=" * 55)  
print("PREDICTED MACHINE STATES")  
print("=" * 55)  
for i, state in enumerate(hidden_states):  
    if observations[i][0] == 0:  
        observation = "Normal"  
    elif observations[i][0] == 1:
```

```

        observation = "High"
    else:
        observation = "Critical"
    print(f"Time {i+1}: Sensor = {observation:8s} → Machine State = {states[state]}")
# =====
# 9. FINAL MACHINE CONDITION
# =====
final_state = hidden_states[-1]
print("\n" + "=" * 55)
print("FINAL MACHINE CONDITION")
print("=" * 55)
print("Current State:", states[final_state])
# =====
# 10. MAINTENANCE RECOMMENDATION
# =====
print("\n" + "=" * 55)
print("MAINTENANCE RECOMMENDATION")
print("=" * 55)
if final_state == 0:
    print("Machine Status: HEALTHY")
    print("Recommendation: Continue normal operation.")
elif final_state == 1:
    print("Machine Status: WARNING")
    print("Recommendation: Schedule preventive maintenance.")
else:
    print("Machine Status: FAILURE")
    print("Recommendation: Stop machine and perform immediate maintenance.")

```

Output:

```

166.0/166.0 kB 5.7 MB/s eta 0:00:00
Libraries imported successfully!

Hidden States:
0 = Healthy
1 = Warning
2 = Failure

Sensor Observation Sequence:
Normal → Normal → High → High → Critical → Critical

=====
PREDICTED MACHINE STATES
=====
Time 1: Sensor = Normal → Machine State = Healthy
Time 2: Sensor = Normal → Machine State = Healthy
Time 3: Sensor = High → Machine State = Warning
Time 4: Sensor = High → Machine State = Warning
Time 5: Sensor = Critical → Machine State = Failure
Time 6: Sensor = Critical → Machine State = Failure

=====
FINAL MACHINE CONDITION
=====
Current State: Failure

=====
MAINTENANCE RECOMMENDATION
=====
Machine Status: FAILURE
Recommendation: Stop machine and perform immediate maintenance.

```

Interpretation:

The sensor observations initially show **normal conditions**. Therefore, the HMM predicts that the machine is **Healthy**.

When the sensor readings become **High**, the predicted state changes to **Warning**.

When the readings become **Critical**, the model predicts **Failure**.

How HMM Supports Predictive Maintenance:

1. Detects hidden machine conditions

The actual machine condition cannot always be directly measured. HMM estimates the hidden condition from sensor data.

2. Early warning

The **Warning** state allows engineers to identify problems before complete equipment failure.

3. Reduces downtime

Maintenance can be scheduled before unexpected machine breakdown.

4. Reduces maintenance cost

Instead of repairing machines only after failure, organizations can perform **preventive maintenance**.

5. Continuous monitoring

Sensor data can be continuously supplied to the HMM to monitor machine health.

Conclusion:

A **Hidden Markov Model** is useful for predictive machine maintenance because it estimates hidden machine conditions from observable sensor readings. In this example, the machine moves from **Healthy** → **Warning** → **Failure** as sensor conditions become increasingly abnormal. This helps manufacturing organizations identify potential failures early, schedule maintenance, reduce unexpected downtime, and improve equipment reliability.

QUESTION-3:

Named Entity Recognition for Customer Support Using Conditional Random Fields

Industry Scenario:

A customer-support company receives thousands of emails every day.

The company wants to automatically identify important information such as:

- **Customer Name**
- **Product Name**
- **Order ID**
- **Issue Type**

For example:

"John ordered a Dell Laptop with order ID ORD12345.

The laptop is not working."

The CRF should identify:

John → CUSTOMER_NAME

Dell → PRODUCT_NAME

Laptop → PRODUCT_NAME

ORD12345 → ORDER_ID

not working → ISSUE_TYPE

CRF Structure:

A Conditional Random Field predicts labels by considering the **entire sequence of words**.

Input Customer Email



Tokenization



Word-Level Features



CRF Model

↓
Entity Labels

↓
Customer Name
Product Name

Order ID
Issue Type

Example Training Data

We use BIO-style labels:

Word	Label
John	B-CUSTOMER
ordered	O
Dell	B-PRODUCT
Laptop	I-PRODUCT
with	O
order	O
ID	O
ORD12345	B-ORDER
The	O
laptop	B-PRODUCT
is	O
not	B-ISSUE
working	I-ISSUE

Where:

- **B** = Beginning of entity
- **I** = Inside entity
- **O** = Other

Code:

```
0 # =====  
# NAMED ENTITY RECOGNITION USING CONDITIONAL RANDOM FIELD  
# =====  
# pip -q install sklearn-crfsuite  
import sklearn_crfsuite  
from sklearn_crfsuite import metrics  
print("Libraries imported successfully!")  
# =====  
# 1. TRAINING DATA  
# =====  
train_sentences = [  
    ["John", "B-CUSTOMER"), ("ordered", "O"), ("Dell", "B-PRODUCT"), ("Laptop", "I-PRODUCT"), ("with", "O"), ("order", "O"), ("ID", "O"), ("anc  
    ["Mary", "B-CUSTOMER"), ("bought", "O"), ("HP", "B-PRODUCT"), ("Printer", "I-PRODUCT"), ("order", "O"), ("ORD45678", "B-ORDER"), ("anc  
    ["David", "B-CUSTOMER"), ("purchased", "O"), ("Apple", "B-PRODUCT"), ("iPhone", "I-PRODUCT"), ("order", "O"), ("ORD78901", "B-ORDER")  
    ["Sarah", "B-CUSTOMER"), ("ordered", "O"), ("Lenovo", "B-PRODUCT"), ("Laptop", "I-PRODUCT"), ("order", "O"), ("ORD23456", "B-ORDER")  
    ["Robert", "B-CUSTOMER"), ("purchased", "O"), ("Samsung", "B-PRODUCT"), ("Phone", "I-PRODUCT"), ("order", "O"), ("ORD56789", "B-ORDER")  
]  
# =====  
# 2. FEATURE EXTRACTION  
# =====  
def word_features(sentence, index):  
    word = sentence[index][0]  
    features = [  
        "word.lower()": word.lower(),  
        "word[-3:]": word[-3:],  
        "word[-2:]": word[-2:],  
        "word.isupper()": word.isupper(),  
        "word.istitle()": word.istitle(),  
        "word.isdigit()": word.isdigit(),  
        "word.length": len(word)  
    ]  
    if index > 0:  
        previous_word = sentence[index-1][0]  
        features.update({  
            "-1:word.lower()": previous_word.lower(),  
            "-1:word.istitle()": previous_word.istitle()  
        })  
    else:  
        features["BOS"] = True  
    if index < len(sentence)-1:  
        next_word = sentence[index+1][0]  
        features.update({  
            "+1:word.lower()": next_word.lower(),  
            "+1:word.istitle()": next_word.istitle()  
        })  
    else:  
        features["EOS"] = True  
    return features  
def sentence_features(sentence):  
    return [word_features(sentence, i) for i in range(len(sentence))]  
# =====  
# 3. PREPARE TRAINING DATA  
# =====  
X_train = [sentence_features(sentence) for sentence in train_sentences]  
y_train = [label for word, label in sentence] for sentence in train_sentences]  
print("\nTraining data prepared successfully!")  
# =====  
# 4. CREATE CRF MODEL  
# =====
```

```

# =====
crf=sklearn_crfsuite.CRF(
    algorithm="lbfgs",
    c1=0.1,
    c2=0.1,
    max_iterations=100,
    all_possible_transitions=True
)

# =====
# 5. TRAIN CRF MODEL
# =====
crf.fit(X_train,y_train)
print("CRF model trained successfully!")
# =====
# 6. TEST DATA
# =====
test_sentence=[
    ("Michael",),
    ("ordered",),
    ("Dell",),
    ("Laptop",),
    ("with",),
    ("order",),
    ("ID",),
    ("ORD99999",),
    ("and",),
    ("the",),
    ("laptop",),
    ("is",),
    ("not",),
    ("working")
]
test_sentence=[(word[0],"O") for word in test_sentence]
# =====
# 7. EXTRACT TEST FEATURES
# =====
X_test=[sentence_features(test_sentence)]
# =====
# 8. PREDICT ENTITY LABELS
# =====
prediction=crf.predict(X_test)[0]
# =====
# 9. DISPLAY PREDICTIONS
# =====
print("\n"+"="*60)
print("NAMED ENTITY RECOGNITION RESULTS")
print("="*60)
for i,word in enumerate(test_sentence):
    print(f"{word[0]:15s} → {prediction[i]}")
# =====
# 10. DISPLAY EXTRACTED ENTITIES
# =====
print("\n"+"="*60)
print("EXTRACTED ENTITIES")
print("="*60)
current_entity=[]
current_label=None
for i,label in enumerate(prediction):
    word=test_sentence[i][0]
    if label.startswith("B-"):
        if current_entity:
            print(current_label,":", " ".join(current_entity))
            current_entity=[word]
            current_label=label[2:]
        elif label.startswith("I-"):
            current_entity.append(word)
        else:
            if current_entity:
                print(current_label,":", " ".join(current_entity))
                current_entity=[]
                current_label=None
    if current_entity:
        print(current_label,":", " ".join(current_entity))

```

Output:

```
... 1.2/1.2 MB 22.4 MB/s eta 0:00:00
Libraries imported successfully!

Training data prepared successfully!
CRF model trained successfully!

=====
NAMED ENTITY RECOGNITION RESULTS
=====
Michael      → B-CUSTOMER
ordered      → O
Dell         → B-PRODUCT
Laptop       → I-PRODUCT
with         → O
order        → O
ID           → O
ORD99999     → B-ORDER
and          → O
the          → O
laptop       → B-PRODUCT
is           → O
not          → B-ISSUE
w            → I-ISSUE

=====
EXTRACTED ENTITIES
=====
CUSTOMER : Michael
PRODUCT  : Dell Laptop
ORDER    : ORD99999
PRODUCT  : laptop
ISSUE    : not w
```

How CRFs Improve CRM Systems:

1. Automatic information extraction

CRFs automatically identify important information from customer emails.

Customer Name → Michael

Product → Dell Laptop

Order ID → ORD99999

Issue → not working

2. Understands word context

CRFs consider neighboring words when assigning labels.

For example:

Dell Laptop

can be recognized as a product entity because the model considers the surrounding sequence.

3. Better customer support

The extracted information can automatically be sent to CRM systems.

Customer Email



CRF NER



Customer / Product / Order / Issue



CRM System



Automatic Ticket Creation

4. Reduces manual work

Employees do not need to manually read thousands of emails and extract order numbers or issue types.

5. Faster ticket routing

The detected issue can be used to automatically route the customer request to the appropriate support team.

Conclusion:

A Conditional Random Field (CRF) is an effective sequence-labeling model for Named Entity Recognition. It can identify important entities such as Customer Name, Product Name, Order ID, and Issue Type from customer-support emails. By automatically extracting this information, CRFs can improve CRM systems, reduce manual data entry, accelerate ticket processing, and provide faster customer support.